Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)
[Course Code: 23ECSPE1AD]

Submitted by

Abhishek Adagonda Majage **1BM25EC402**

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering

Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

AAT – VERILOG HDL DESIGN AND SIMULATION

Activity / Assessment Task Record

Submission: Design, Verilog HDL source code, testbench, and simulation result for each listed module. Execute one combinational and one sequential module using the Cadence environment available in the laboratory.

General Procedure

1. Create the design module and save it with a .v extension.
2. Create the corresponding testbench in a separate file or below the design module.
3. Compile the design and testbench in the simulator. Select the testbench module as the simulation top.
4. Run the simulation and inspect the printed values and/or waveform.
5. For the Cadence demonstration, show the input signals, clock/reset where applicable, and output waveform.

1. Arithmetic Logic Unit (ALU)

Aim: To design and simulate a 4-bit ALU performing addition, subtraction, AND, OR, XOR, NOT, left shift, and right shift.

Design: Inputs A[3:0], B[3:0], select SEL[2:0]; outputs Y[3:0] and carry. SEL selects the operation.

Verilog HDL Code

```
module alu4(input [3:0] a, b, input [2:0] sel, output reg [3:0] y, output reg carry);
always @* begin
  y = 4'b0000;
  carry = 1'b0;
  case (sel)
    3'b000: {carry,y} = {1'b0,a} + {1'b0,b};
    3'b001: y = a - b;
    3'b010: y = a & b;
    3'b011: y = a | b;
    3'b100: y = a ^ b;
    3'b101: y = ~a;
    3'b110: y = a << 1;
    3'b111: y = a >> 1;
    default: begin y = 0; carry = 0; end
  endcase
end
endmodule
```

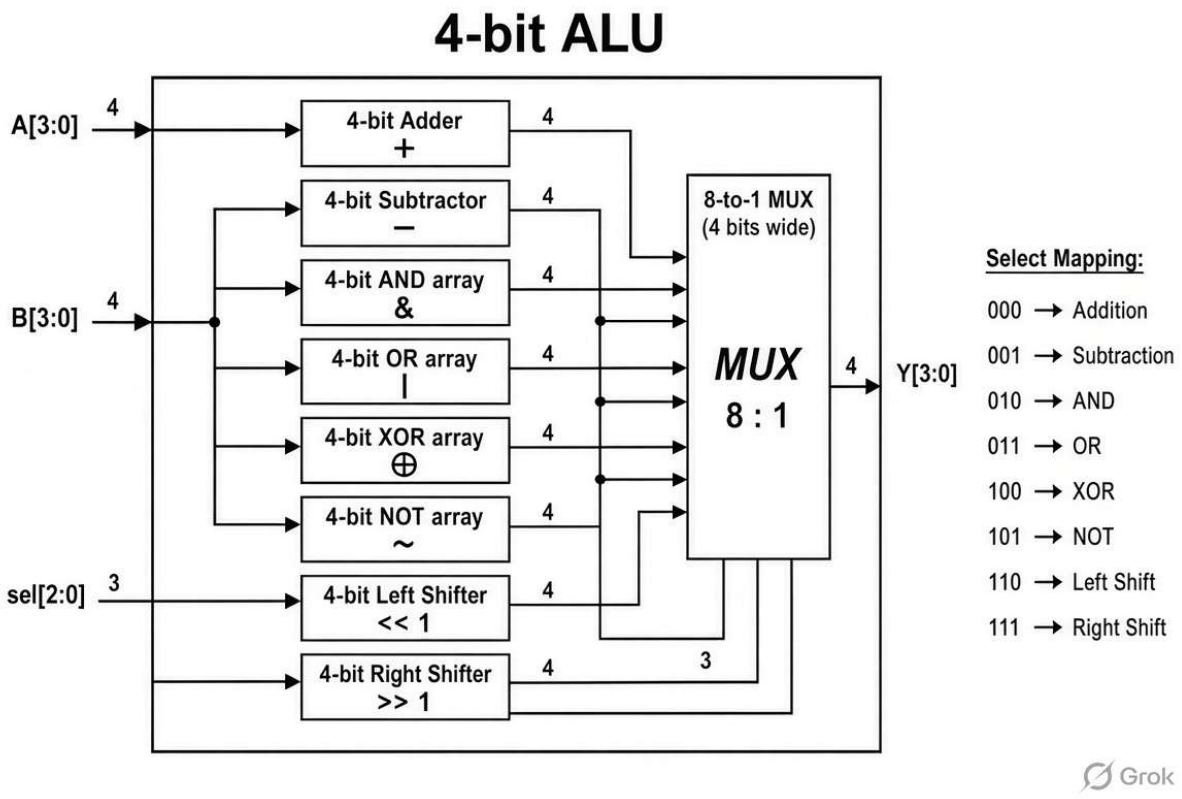
Testbench (TB)

```
`timescale 1ns/1ps
module tb_alu4;
reg [3:0] a,b;
reg [2:0] sel;
wire [3:0] y;
wire carry;
alu4 dut(.a(a),.b(b),.sel(sel),.y(y),.carry(carry));
initial begin
  a=7; b=3;
  sel=0; #10; sel=1; #10; sel=2; #10; sel=3; #10;
  sel=4; #10; sel=5; #10; sel=6; #10; sel=7; #10;
  $finish;
end
initial $monitor("t=%0t a=%d b=%d sel=%b y=%d carry=%b",$time,a,b,sel,y,carry);
endmodule
```

Procedure

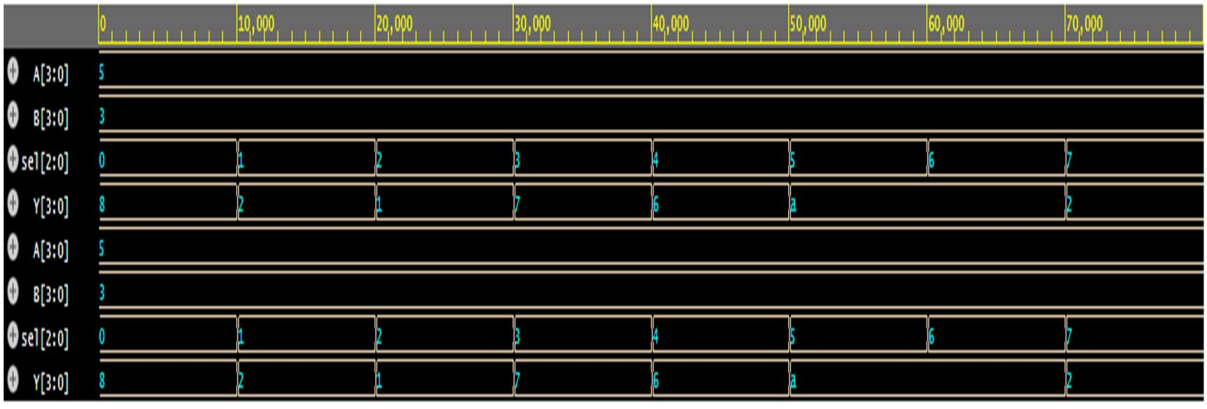
Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

Circuit Diagrams:



Expected Output / Result

Expected result: The output changes according to the selected arithmetic or logical operation.



2. 4-Bit Comparator

Aim: To compare two 4-bit binary numbers.

Design: Inputs A and B; outputs EQ=1 when equal, GT=1 when A>B, and LT=1 when A<B.

Verilog HDL Code

```
module comparator4(input [3:0] a,b, output eq,gt,lt);
assign eq = (a == b);
assign gt = (a > b);
assign lt = (a < b);
endmodule
```

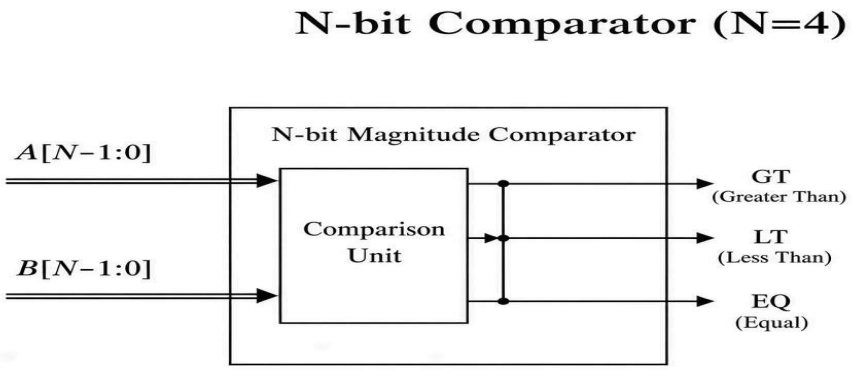
Testbench (TB)

```
`timescale 1ns/1ps
module tb_comparator4;
reg [3:0] a,b;
wire eq,gt,lt;
comparator4 dut(.a(a),.b(b),.eq(eq),.gt(gt),.lt(lt));
initial begin
  a=4; b=4; #10;
  a=9; b=2; #10;
  a=1; b=7; #10;
  $finish;
end
initial $monitor("a=%d b=%d EQ=%b GT=%b LT=%b",a,b,eq,gt,lt);
endmodule
```

Procedure

Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

Circuit Diagrams:

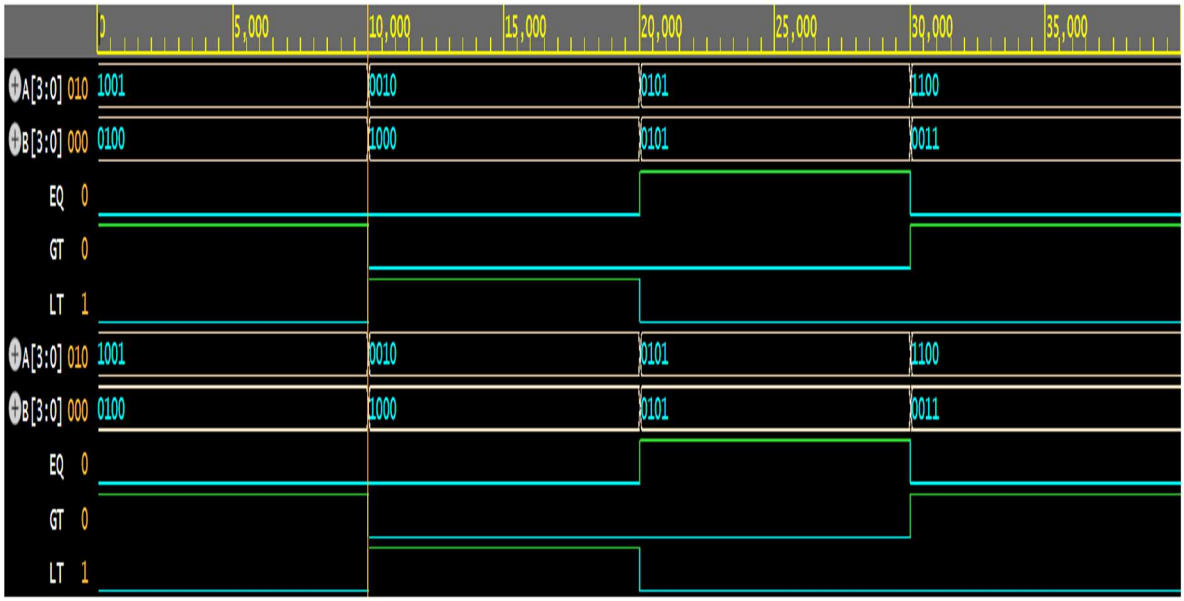


Comparison Condition	Outputs
If $A > B$	GT = 1 LT = 0 EQ = 0
If $A < B$	GT = 0 LT = 1 EQ = 0
If $A = B$	GT = 0 LT = 0 EQ = 1

Grok

Expected Output / Result

Expected result: Exactly one of EQ, GT, or LT is high for known binary inputs.



3. 4-Bit Parity Generator and Checker

Aim: To generate even/odd parity and check a received 5-bit word.

Design: The generator creates a parity bit for four data bits. The checker XORs all four received data bits and the received parity bit; for even parity, XOR result 0 means no error.

Verilog HDL Code

```
module parity4(input [3:0] data, output even_parity, odd_parity);
assign even_parity = ^data;
assign odd_parity  = ~(^data);
endmodule

module parity_checker5(input [4:0] received, output parity_error);
assign parity_error = ^received; // 1 means even-parity error
endmodule
```

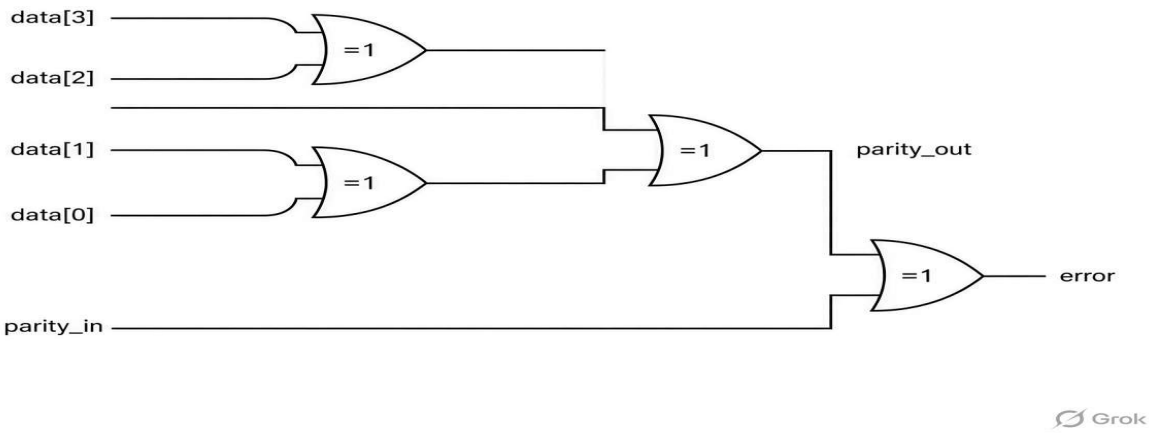
Testbench (TB)

```
`timescale 1ns/1ps
module tb_parity4;
reg [3:0] data;
wire even_parity,odd_parity;
reg [4:0] received;
wire parity_error;
parity4 gen(.data(data),.even_parity(even_parity),.odd_parity(odd_parity));
parity_checker5 chk(.received(received),.parity_error(parity_error));
initial begin
  data=4'b0000; #10;
  data=4'b1011; #10;
  received=5'b10110; #10; // even number of 1s: no error
  received=5'b10111; #10; // odd number of 1s: error
  $finish;
end
initial $monitor("data=%b even=%b odd=%b received=%b error=%b",
                 data,even_parity,odd_parity,received,parity_error);
endmodule
```

Procedure

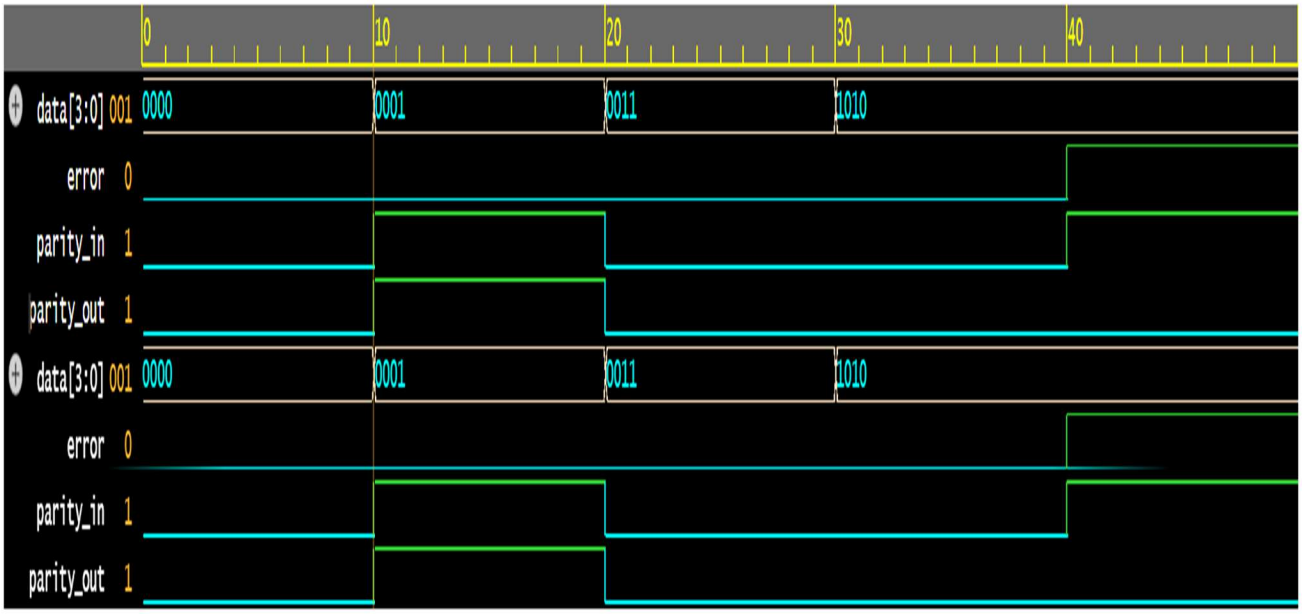
Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

Circuit Diagrams:



Expected Output / Result

Expected result: even_parity is the bit needed to make the total number of ones even. parity_error=1 indicates a parity error.



4. 4 Bit Barrel Shifter -

Aim: To shift a 4-bit input left or right by a selectable number of bit positions.

Design: DIR=0 selects left shift; DIR=1 selects right shift. SHAMT specifies the shift amount.

Verilog HDL Code

```
module barrel_shifter4(input [3:0] data, input [1:0] shamt,
                       input dir, output reg [3:0] y);
always @* begin
  if (dir == 1'b0) y = data << shamt;
  else             y = data >> shamt;
end
endmodule
```

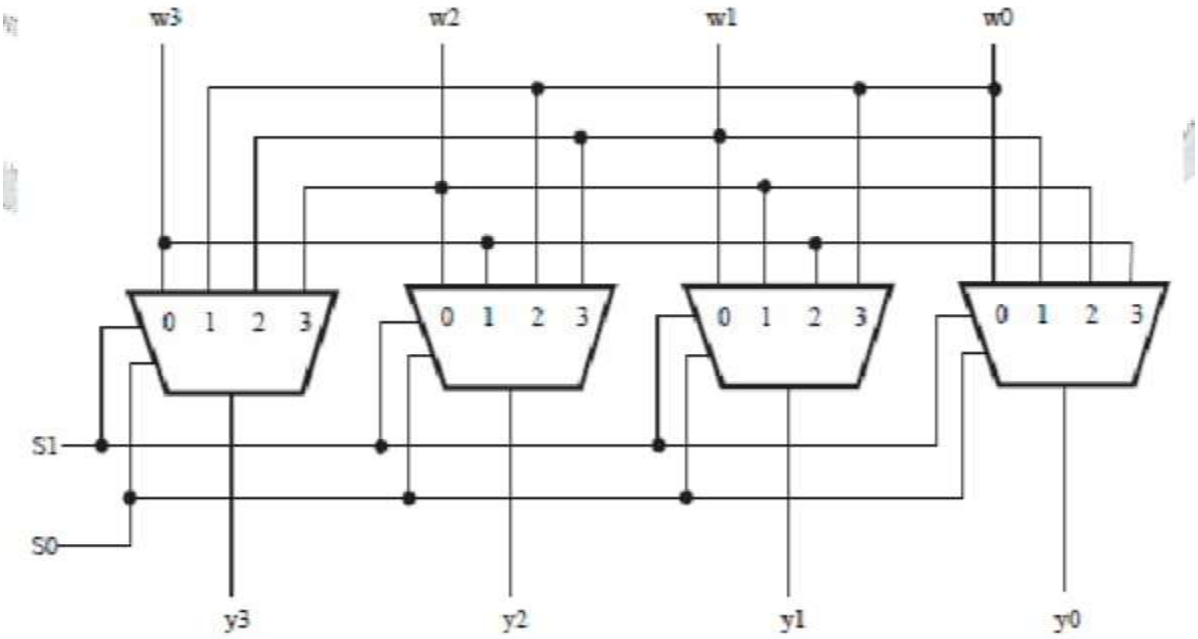
Testbench (TB)

```
`timescale 1ns/1ps
module tb_barrel_shifter4;
reg [3:0] data;
reg [1:0] shamt;
reg dir;
wire [3:0] y;
barrel_shifter4 dut(.data(data),.shamt(shamt),.dir(dir),.y(y));
initial begin
  data=4'b1011; dir=0; shamt=0; #10;
  shamt=1; #10; shamt=2; #10;
  dir=1; shamt=1; #10; shamt=2; #10;
  $finish;
end
initial $monitor("data=%b dir=%b shamt=%d y=%b",data,dir,shamt,y);
endmodule
```

Procedure

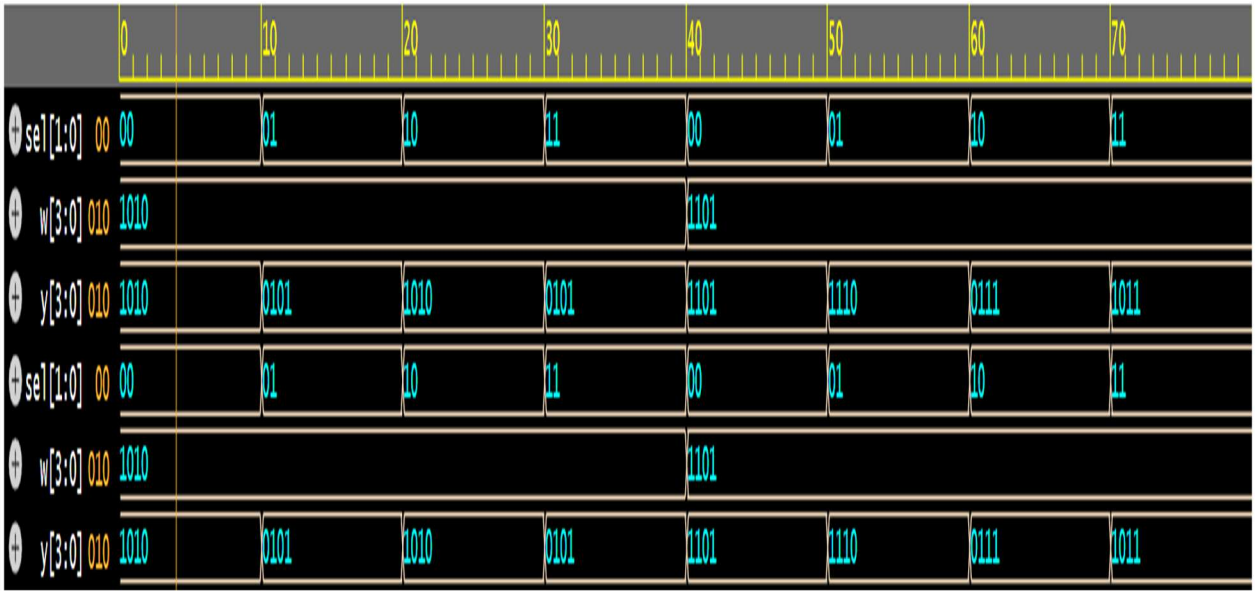
Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

Circuit Diagrams:



Expected Output / Result

Expected result: Y displays the selected logical shift, with shifted-out bits discarded.



5. Timer Module

Aim: To design a parameterized timer that counts clock cycles and asserts DONE at its terminal count.

Design: The counter increments when ENABLE=1. At TERMINAL, it clears the count and pulses DONE for one clock cycle.

Verilog HDL Code

```
module timer #(parameter WIDTH=8, parameter TERMINAL=8'd99)
(input clk, rst, enable, output reg done, output reg [WIDTH-1:0] count);
always @(posedge clk or posedge rst) begin
  if (rst) begin count <= 0; done <= 0; end
  else if (enable) begin
    if (count == TERMINAL) begin count <= 0; done <= 1; end
    else begin count <= count + 1'b1; done <= 0; end
  end else done <= 0;
end
endmodule
```

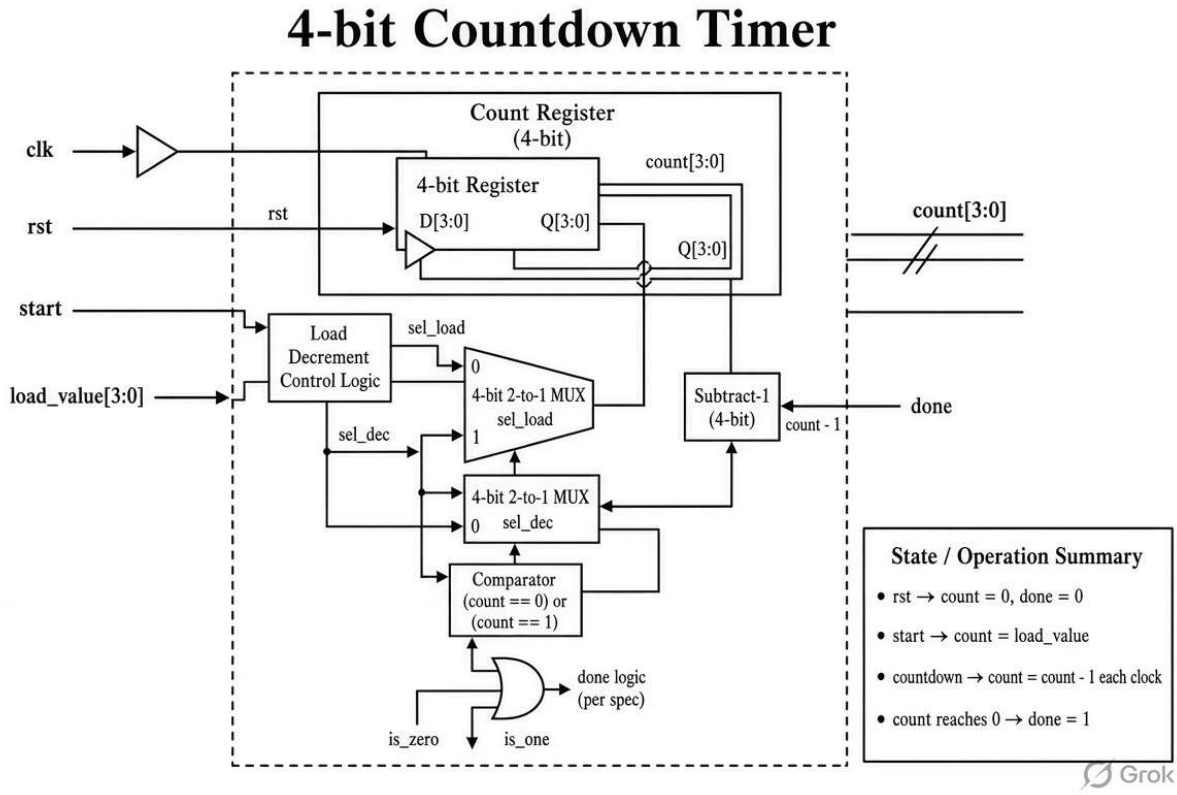
Testbench (TB)

```
`timescale 1ns/1ps
module tb_timer;
reg clk=0, rst=1, enable=0;
wire done;
wire [7:0] count;
timer #(.WIDTH(8),.TERMINAL(8'd4)) dut(.clk(clk),.rst(rst),.enable(enable),.done(done),.count(count));
always #5 clk=~clk;
initial begin #12; rst=0; enable=1; #100; $finish; end
initial $monitor("t=%0t count=%d done=%b",$time,count,done);
endmodule
```

Procedure

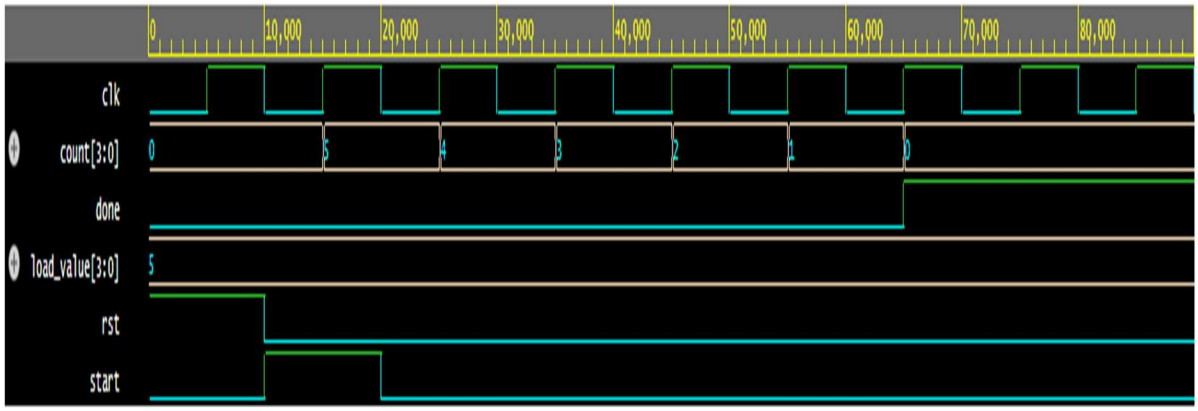
Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

Circuit Diagrams:



Expected Output / Result

Expected result: Count advances on rising clock edges and returns to zero after reaching the terminal count.



6. Ring Counter

Aim: To design a 4-bit ring counter.

Design: A single 1 circulates through the four flip-flop positions. Reset initializes Q=0001.

Verilog HDL Code

```
module ring_counter4(input clk, rst, output reg [3:0] q);
always @(posedge clk or posedge rst) begin
  if (rst) q <= 4'b0001;
  else q <= {q[2:0],q[3]};
end
endmodule
```

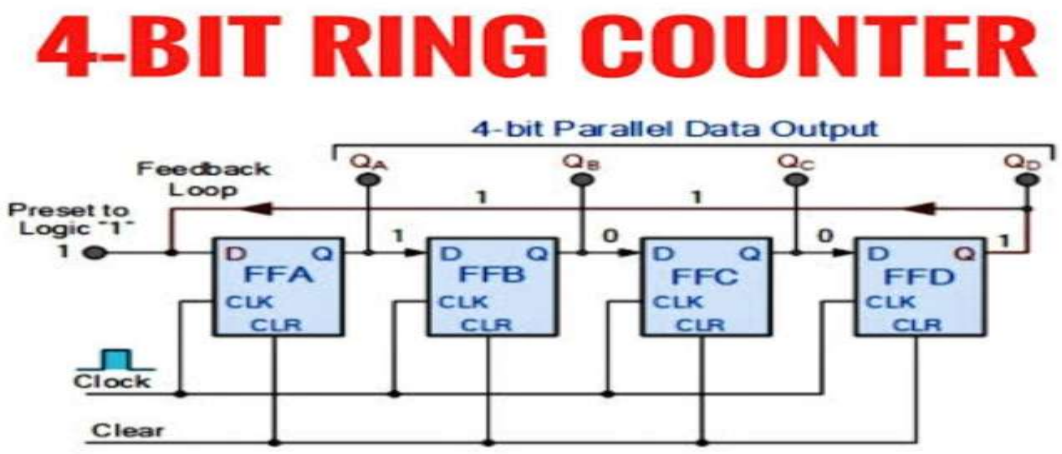
Testbench (TB)

```
`timescale 1ns/1ps
module tb_ring_counter4;
reg clk=0, rst=1;
wire [3:0] q;
ring_counter4 dut(.clk(clk),.rst(rst),.q(q));
always #5 clk=~clk;
initial begin #12; rst=0; #80; $finish; end
initial $monitor("t=%0t q=%b",$time,q);
endmodule
```

Procedure

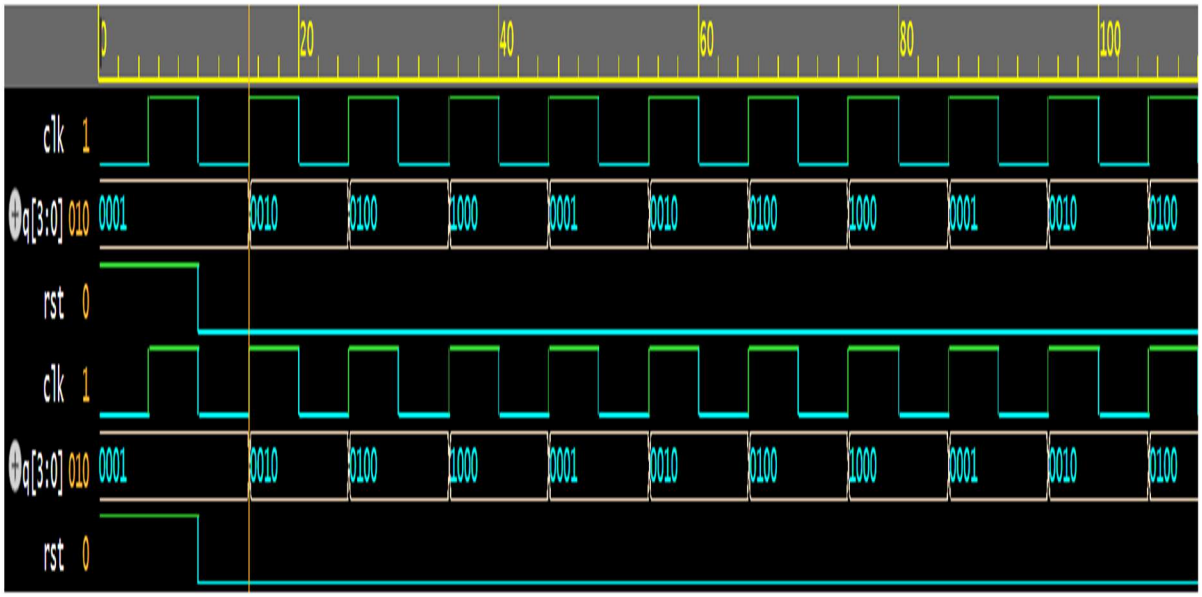
Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

Circuit Diagrams:



Expected Output / Result

Expected result: The output cycles through 0001, 0010, 0100, 1000, then repeats.



7. Johnson Counter

Aim: To design a 4-bit Johnson counter.

Design: The complemented last bit is fed back to the first stage. A 4-bit Johnson counter has up to eight states.

Verilog HDL Code

```
module johnson_counter4(input clk, rst, output reg [3:0] q);
always @(posedge clk or posedge rst) begin
  if (rst) q <= 4'b0000;
  else q <= {q[2:0],~q[3]};
end
endmodule
```

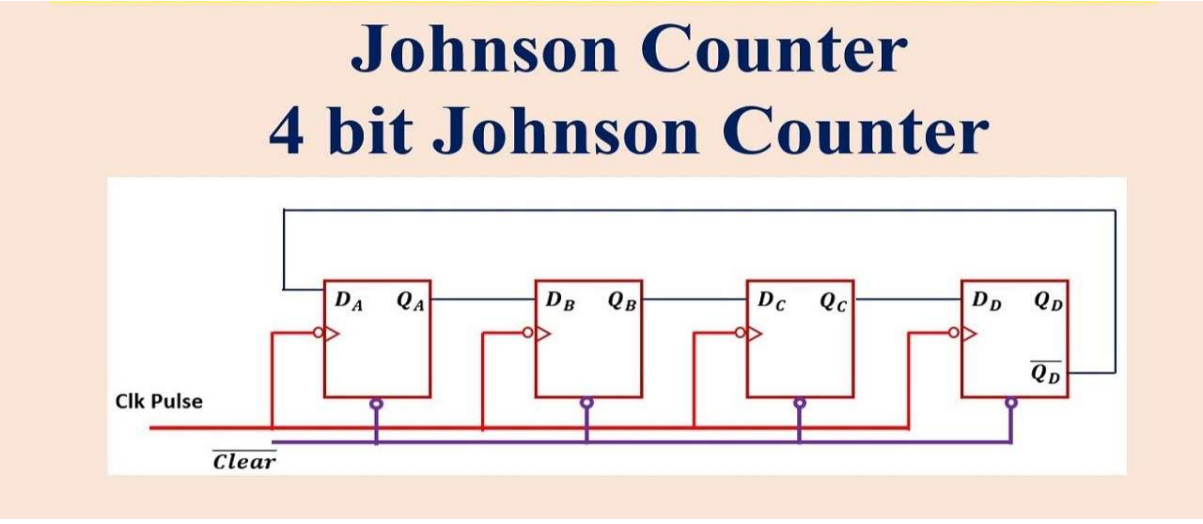
Testbench (TB)

```
`timescale 1ns/1ps
module tb_johnson_counter4;
reg clk=0, rst=1;
wire [3:0] q;
johnson_counter4 dut(.clk(clk),.rst(rst),.q(q));
always #5 clk=~clk;
initial begin #12; rst=0; #100; $finish; end
initial $monitor("t=%0t q=%b",$time,q);
endmodule
```

Procedure

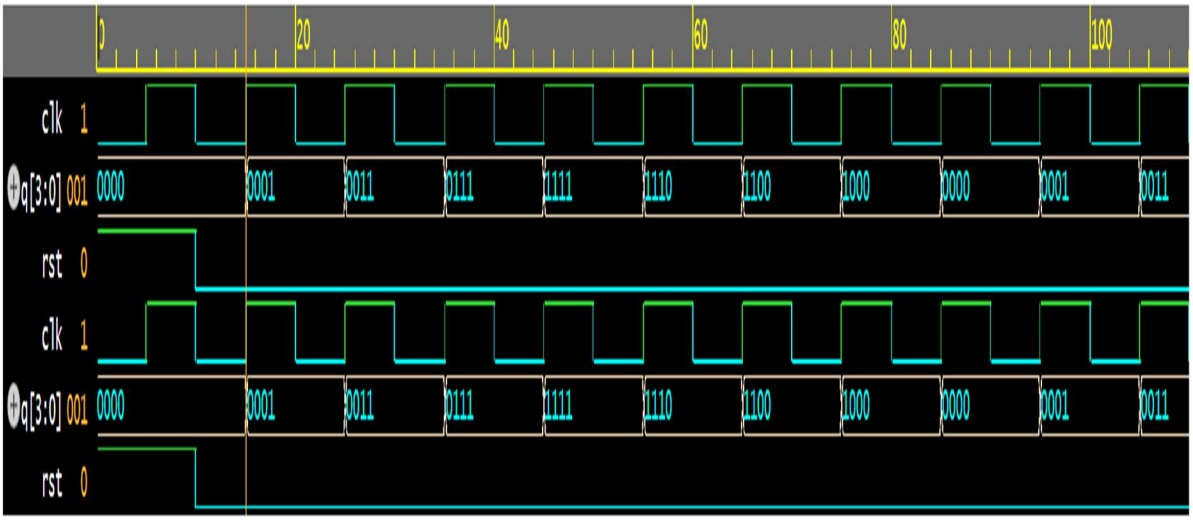
Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

Circuit Diagrams:



Expected Output / Result

Expected result: The sequence is 0000, 0001, 0011, 0111, 1111, 1110, 1100, 1000, then repeats.


0
20
40
60
80
100
clk 1
q[3:0] 001
0000
0001
0011
0111
1111
1110
1100
1000
0000
0001
0011
rst 0
clk 1
q[3:0] 001
0000
0001
0011
0111
1111
1110
1100
1000
0000
0001
0011
rst 0

8. Synchronous and Asynchronous Counters

Aim: To design 4-bit synchronous and asynchronous binary up counters.

Design: In the synchronous counter, all flip-flops use the same clock. In the asynchronous ripple counter, each later stage is clocked by the previous stage output.

Verilog HDL Code

```
module sync_counter4(input clk,rst,en, output reg [3:0] q);
always @(posedge clk or posedge rst) begin
  if (rst) q <= 4'b0000;
  else if (en) q <= q + 1'b1;
end
endmodule

module async_counter4(input clk,rst, output reg [3:0] q);
always @(posedge clk or posedge rst)
  if (rst) q[0] <= 0; else q[0] <= ~q[0];
always @(negedge q[0] or posedge rst)
  if (rst) q[1] <= 0; else q[1] <= ~q[1];
always @(negedge q[1] or posedge rst)
  if (rst) q[2] <= 0; else q[2] <= ~q[2];
always @(negedge q[2] or posedge rst)
  if (rst) q[3] <= 0; else q[3] <= ~q[3];
endmodule
```

Testbench (TB)

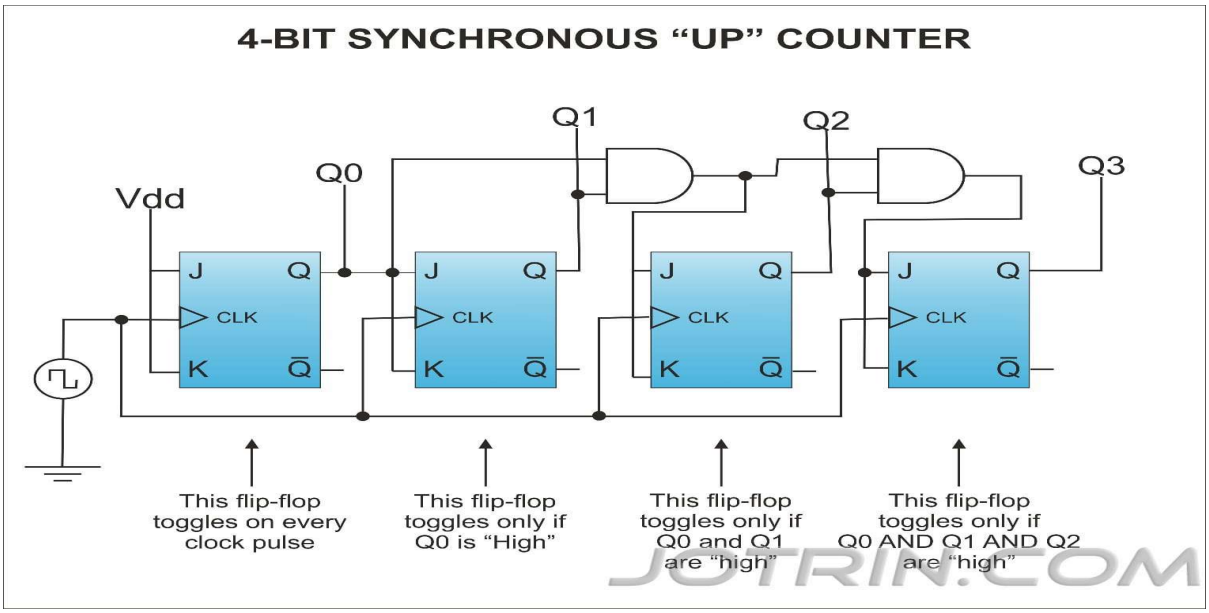
```
`timescale 1ns/1ps
module tb_sync_counter4;
reg clk=0,rst=1,en=1;
wire [3:0] q;
sync_counter4 dut(.clk(clk),.rst(rst),.en(en),.q(q));
always #5 clk=~clk;
initial begin #12; rst=0; #180; $finish; end
initial $monitor("SYNC t=%0t q=%d",$time,q);
endmodule

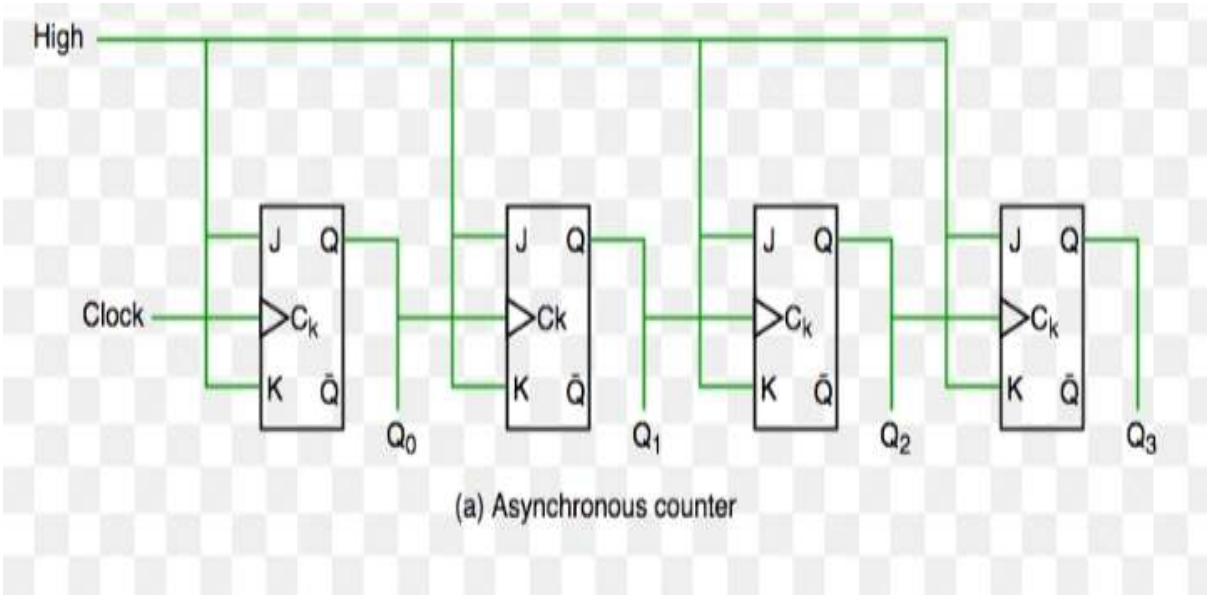
module tb_async_counter4;
reg clk=0,rst=1;
wire [3:0] q;
async_counter4 dut(.clk(clk),.rst(rst),.q(q));
always #5 clk=~clk;
initial begin #12; rst=0; #180; $finish; end
initial $monitor("ASYNC t=%0t q=%d",$time,q);
endmodule
```

Procedure

Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

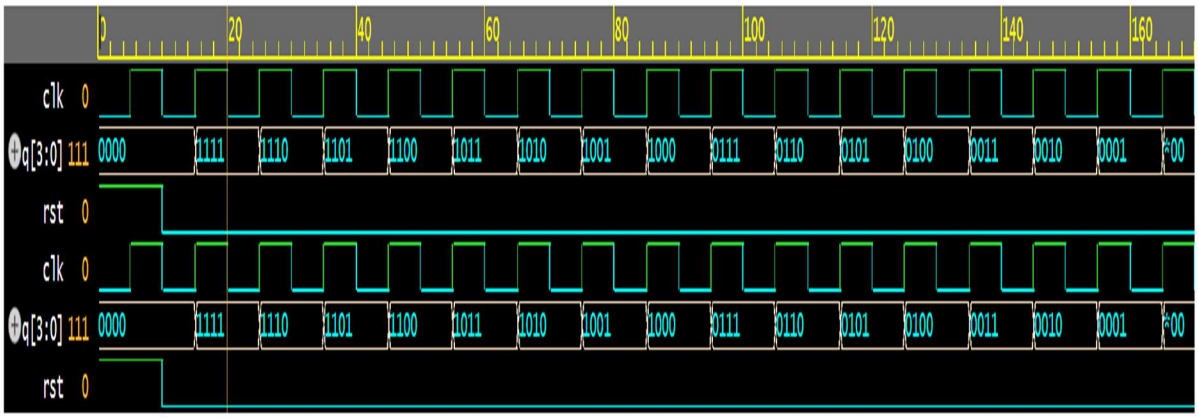
Circuit Diagrams:

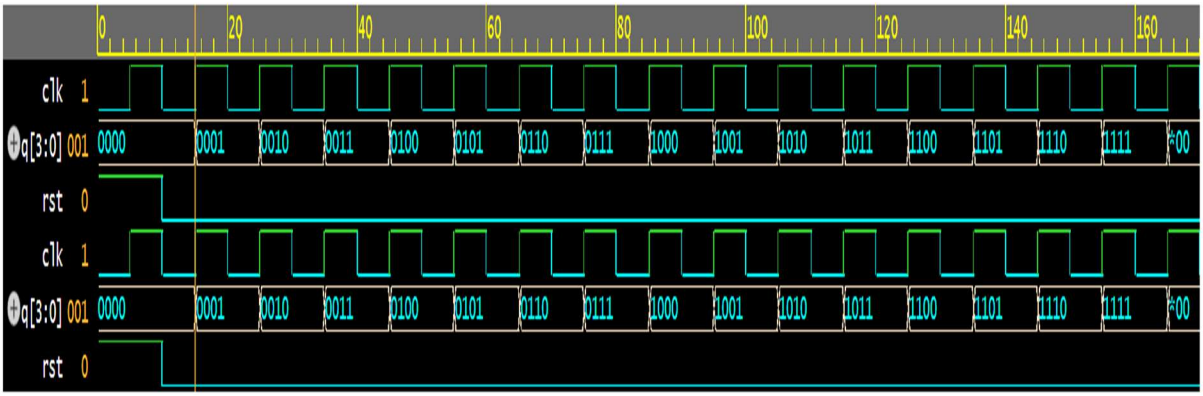




Expected Output / Result

Expected result: Both count in binary from 0 to 15 and wrap to 0. The asynchronous counter has ripple propagation delays in physical hardware.


0
20
40
60
80
100
120
140
160
clk 0
q[3:0] 111
0000
1111
1110
1101
1100
1011
1010
1001
1000
0111
0110
0101
0100
0011
0010
0001
rst 0
clk 0
q[3:0] 111
0000
1111
1110
1101
1100
1011
1010
1001
1000
0111
0110
0101
0100
0011
0010
0001
rst 0


0
20
40
60
80
100
120
140
160
clk 1
q[3:0] 001
0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111
rst 0
clk 1
q[3:0] 001
0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111
rst 0

9. Shift Register

Aim: To design a 4-bit serial-in shift register.

Design: On each rising clock edge with ENABLE=1, the register shifts left and loads SERIAL_IN into the least significant bit.

Verilog HDL Code

```
module shift_register4(input clk,rst,enable,serial_in,
                       output reg [3:0] q, output serial_out);
assign serial_out = q[3];
always @(posedge clk or posedge rst) begin
  if (rst) q <= 4'b0000;
  else if (enable) q <= {q[2:0],serial_in};
end
endmodule
```

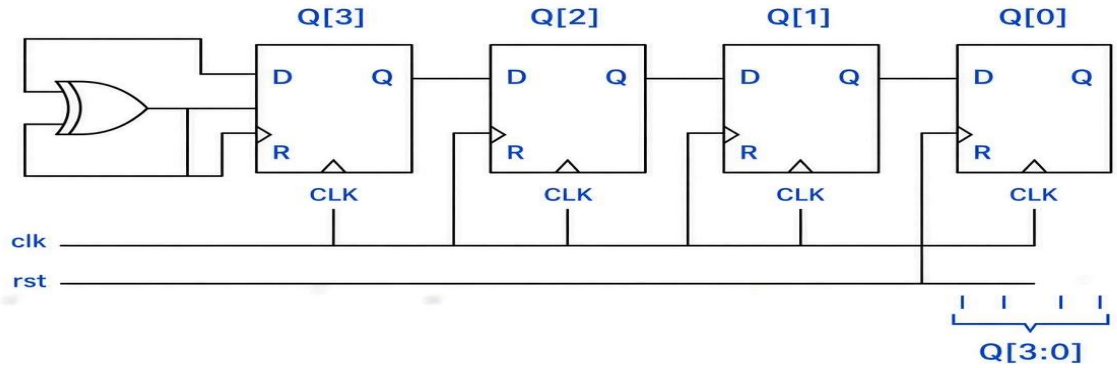
Testbench (TB)

```
`timescale 1ns/1ps
module tb_shift_register4;
reg clk=0,rst=1,enable=1,serial_in=0;
wire [3:0] q;
wire serial_out;
shift_register4 dut(.clk(clk),.rst(rst),.enable(enable),
 .serial_in(serial_in),.q(q),.serial_out(serial_out));
always #5 clk=~clk;
initial begin
  #12; rst=0;
  serial_in=1; #10; serial_in=0; #10;
  serial_in=1; #10; serial_in=1; #40;
  $finish;
end
initial $monitor("t=%0t q=%b serial_out=%b",$time,q,serial_out);
endmodule
```

Procedure

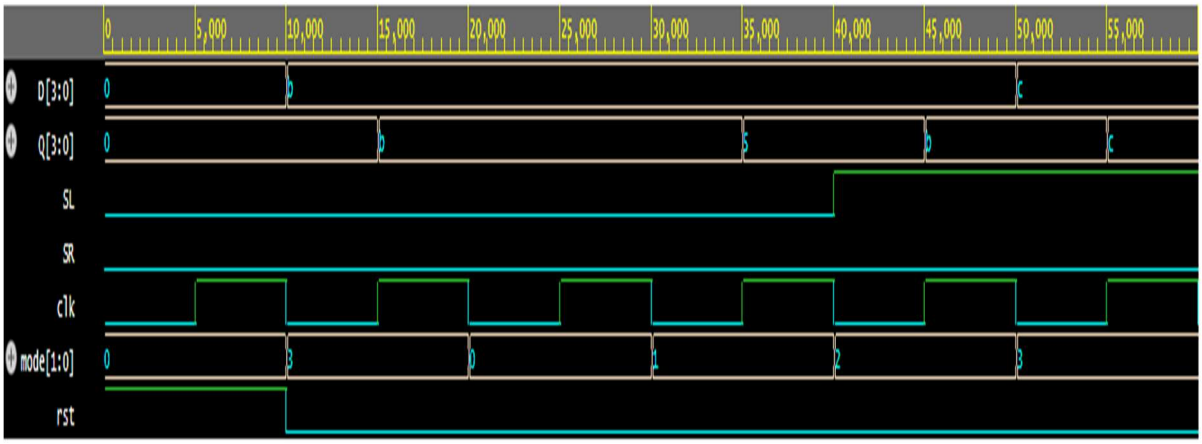
Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

Circuit Diagrams:



Expected Output / Result

Expected result: Serial input bits enter the register one per enabled clock edge; SERIAL_OUT is Q[3].



10. Divide-by-N Counter

Aim: To design a parameterized clock divider.

Design: The internal counter counts N input clock cycles and toggles the divided output. This implementation produces an output frequency of fin/(2N).

Verilog HDL Code

```
module divide_by_n #(parameter N=10, parameter WIDTH=4)
(input clk,rst, output reg clk_div);
reg [WIDTH-1:0] count;
always @(posedge clk or posedge rst) begin
  if (rst) begin count <= 0; clk_div <= 0; end
  else if (count == N-1) begin count <= 0; clk_div <= ~clk_div; end
  else count <= count + 1'b1;
end
endmodule
```

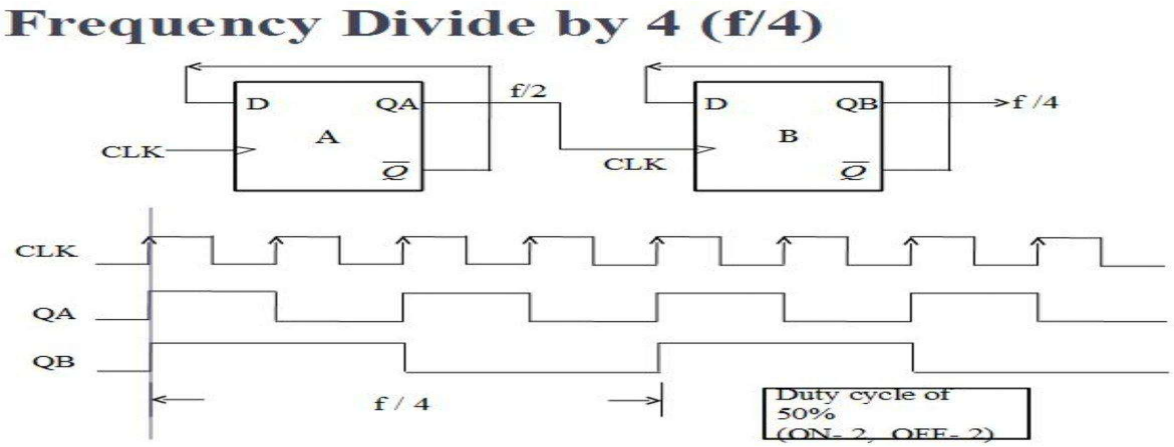
Testbench (TB)

```
`timescale 1ns/1ps
module tb_divide_by_n;
reg clk=0,rst=1;
wire clk_div;
divide_by_n #(.N(3),.WIDTH(2)) dut(.clk(clk),.rst(rst),.clk_div(clk_div));
always #5 clk=~clk;
initial begin #12; rst=0; #150; $finish; end
initial $monitor("t=%0t clk_div=%b",$time,clk_div);
endmodule
```

Procedure

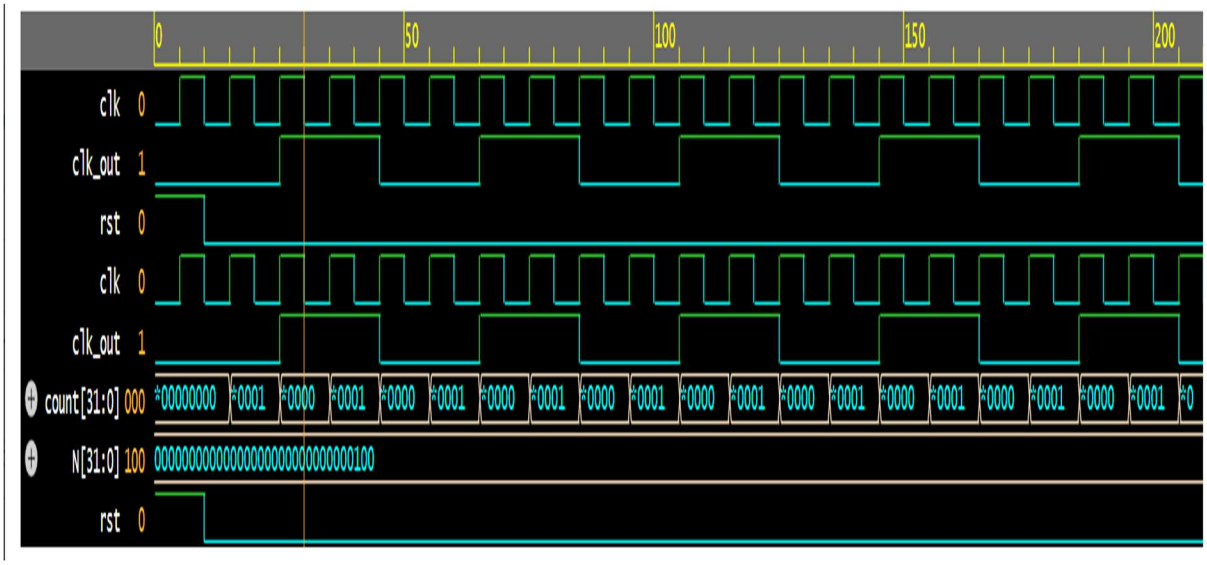
Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

Circuit Diagrams:



Expected Output / Result

Expected result: For N=3, the output toggles every three rising edges; its frequency is one-sixth of the input clock.



11. Mealy and Moore FSMs – Sequence Detector 101

Aim: To design Mealy and Moore finite-state machines that detect the input sequence 101.

Design: The Mealy output depends on the current state and input. The Moore output depends only on the current state.

Verilog HDL Code

```
module mealy_101(input clk,rst,x, output reg z);
localparam S0=2'b00, S1=2'b01, S10=2'b10;
reg [1:0] state,next;
always @(posedge clk or posedge rst)
  if (rst) state <= S0; else state <= next;
always @* begin
  next=state; z=0;
  case(state)
    S0:  if (x) next=S1;
    S1:  if (x) next=S1; else next=S10;
    S10: if (x) begin next=S1; z=1; end else next=S0;
    default: begin next=S0; z=0; end
  endcase
end
endmodule

module moore_101(input clk,rst,x, output z);
localparam S0=3'd0,S1=3'd1,S10=3'd2,S101=3'd3;
reg [2:0] state,next;
always @(posedge clk or posedge rst)
  if (rst) state <= S0; else state <= next;
always @* begin
  next=state;
  case(state)
    S0:   next = x ? S1 : S0;
    S1:   next = x ? S1 : S10;
    S10:  next = x ? S101 : S0;
    S101: next = x ? S1 : S10;
    default: next=S0;
  endcase
end
assign z = (state == S101);
endmodule
```

Testbench (TB)

```
`timescale 1ns/1ps
module tb_mealy_101;
```

```
reg clk=0,rst=1,x=0;
wire z;
mealy_101 dut(.clk(clk),.rst(rst),.x(x),.z(z));
always #5 clk=~clk;
initial begin
  #12; rst=0;
  x=1; #10; x=0; #10; x=1; #10;
  x=0; #10; x=1; #10;
  $finish;
end
initial $monitor("MEALY t=%0t x=%b z=%b",$time,x,z);
endmodule

module tb_moore_101;
reg clk=0,rst=1,x=0;
wire z;
moore_101 dut(.clk(clk),.rst(rst),.x(x),.z(z));
always #5 clk=~clk;
initial begin
  #12; rst=0;
  x=1; #10; x=0; #10; x=1; #10;
  x=0; #10; x=1; #10;
  $finish;
end
initial $monitor("MOORE t=%0t x=%b z=%b",$time,x,z);
endmodule
```

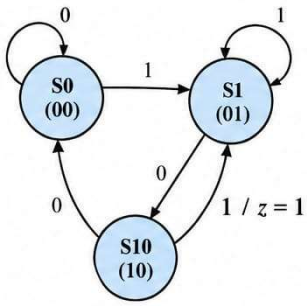
Procedure

Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

Expected Output / Result

Expected result: The Mealy detector asserts its output when the final 1 of 101 is presented. The Moore detector asserts output in its detected state, typically one state transition later.

Mealy Sequence Detector (101)

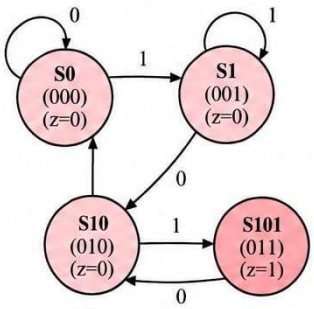


State Diagram (Mealy)

Present State	Input (x)		Output z
	0	1	
S0	S0 / 0	S1 / 0	-
S1	S10 / 0	S1 / 0	-
S10	S0 / 0	S1 / 1	-

State Table (Mealy)

Moore Sequence Detector (101)

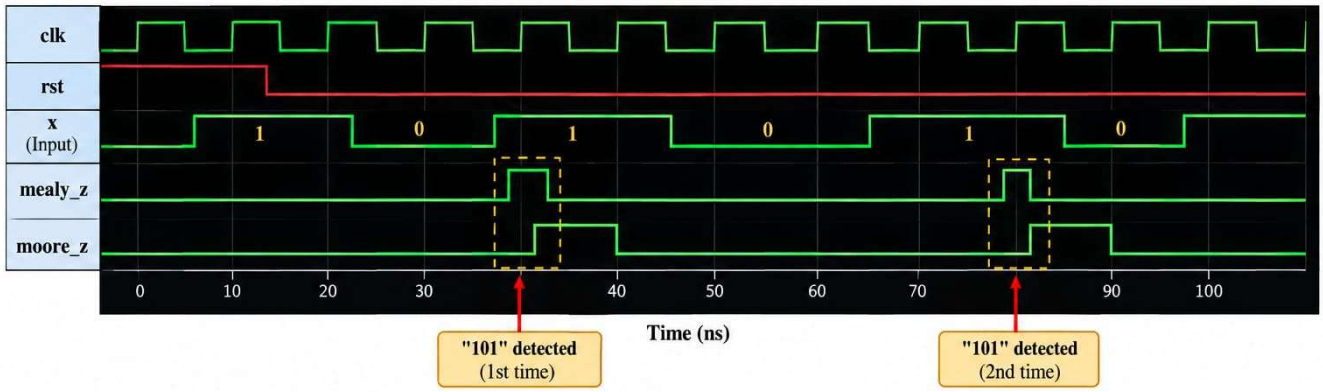


State Diagram (Moore)

Present State	Input (x)		Output z
	0	1	
S0	S0	S1	0
S1	S10	S1	0
S10	S0	S101	0
S101	S10	S1	1

State Table (Moore)

Simulation Waveform (EDA Playground)



12. Random Sequence Generator (4-bit LFSR)

Aim: To generate a repeatable pseudo-random bit sequence using a 4-bit linear feedback shift register.

Design: XOR feedback is formed from Q[3] and Q[2]. Reset initializes the register to a nonzero seed.

Verilog HDL Code

```
module lfsr4(input clk,rst,enable, output reg [3:0] q);
wire feedback = q[3] ^ q[2];
always @(posedge clk or posedge rst) begin
  if (rst) q <= 4'b0001;
  else if (enable) q <= {q[2:0],feedback};
end
endmodule
```

Testbench (TB)

```
`timescale 1ns/1ps
module tb_lfsr4;
reg clk=0,rst=1,enable=1;
wire [3:0] q;
lfsr4 dut(.clk(clk),.rst(rst),.enable(enable),.q(q));
always #5 clk=~clk;
initial begin #12; rst=0; #160; $finish; end
initial $monitor("t=%0t LFSR=%b",$time,q);
endmodule
```

Procedure

Compile the design and testbench, select the testbench module as the simulation top, run the simulation, and observe the output values/waveforms.

Expected Output / Result

Expected result: The register moves through a repeatable sequence of nonzero states. It is pseudo-random, not truly random.

